The Intelligent Worm: Adaptive Malware

A Defensive Threat Analysis through the Reproductive Threshold of Cyber Epidemics

Dendi Suhubdy
Scorpion Labs & Bitwyre Research
dendi@scorpionlabs.ai

July 12, 2026

Abstract

This is a defensive threat-modeling essay in the tradition of Staniford, Paxson, and Weaver's *How to 0wn the Internet in Your Spare Time* (USENIX Security 2002) — a paper that described worm dynamics precisely so that defenders could reason about them. We analyze the hypothetical *Intelligent Worm*: a self-propagating program whose exploitation capability is not fixed at authoring time but regenerated by an onboard reasoning loop. We frame worm dynamics epidemiologically through the basic reproductive number R_0, show why patching has historically driven outbreaks below the epidemic threshold by depleting the susceptible fraction, and argue that an adaptive exploitation term changes the shape of that curve — turning a one-time cliff into a contested equilibrium. We take the feasibility limits seriously, survey what current LLM-agent security research actually demonstrates, examine a maximal capability stack (centralized exploit generation, adaptive pacing, inter-worm coordination, rootkit-class concealment, cross-platform reach) and the detection surface each subsystem necessarily creates, and lay out the defensive re-architecture the threat model implies. There is no exploit code, no propagation implementation, and no operational recipe here: the point is to stress-test the assumptions our current defenses rest on before an adaptive adversary does it for us.

Keywords: computer worms; malware propagation; epidemic modeling; reproductive number; autonomous agents; large language models; threat modeling; behavioral detection; defensive security.

1 The assumption underneath our defenses

The single assumption underneath most of our defensive posture is that **malware is static between releases**. An exploit is discovered, weaponized, and shipped by a human author; defenders reverse it, extract a signature or a patch, and the specific threat decays. The whole signature-and-patch industry — antivirus definitions, IDS rules, CVE-driven patch cycles — is an answer to an adversary whose capabilities are *fixed at authoring time*.

This essay asks what happens to that assumption if the exploitation capability is not fixed but **regenerated by an onboard reasoning loop**. Call the hypothetical result an *Intelligent Worm*. The propagation machinery — scanning, self-replication, persistence — is old and well understood. What changes is that the *infection vector* stops being a finite asset carried from the author and becomes an adaptive capability the program produces on the fly. That one change is best understood not as "a smarter worm" but as a **change in the epidemiology** of the threat.

2 Worms, precisely

A worm is self-replicating malware that spreads across a network *without a human running it.* That autonomy is the whole distinction:

- **Virus** — attaches to a host file or program; spreads when a person executes the infected file.
- **Worm** — a standalone program that copies itself to new hosts over a network; no user action required.
- **Trojan** — disguised as benign; relies on the user being fooled.

Worms are defined by network self-propagation. The security literature decomposes a worm into four functional parts: (1) **target discovery** (the scanner), which sets the scan rate; (2) the **propagation vector**, almost always the exploitation of a vulnerability, and the component the Intelligent Worm proposes to make adaptive; (3) the **payload**, which is independent of propagation and can be nothing at all; and (4) **self-replication and persistence**, the recursion that produces exponential growth.

```
1. Running on an infected host
2. Scan the network for reachable targets
3. For each target: probe for a weakness
4. Exploit it -> gain remote code execution
5. Transfer a copy of itself to the victim
6. Execute the copy -> new host now infected
7. (Optionally) run payload, install persistence
8. New host returns to step 2 -- recursion
```

3 The epidemiological frame is the right one

Worm propagation is not *like* an epidemic; it is modeled with literally the same mathematics. The Slammer analysis [6] and the *How to 0wn the Internet* paper [5] both use the classical susceptible–infected (SI/SIR) framework. The lineage is explicit: the SIR model and the reproductive number come from Kermack and McKendrick's 1927 epidemic theory [1]; Kephart and White first mapped it onto computer viruses [2]; and recent work fits compartmental models directly to real malware traces — Chernikova et al.'s SIIDR model [3] reproduces WannaCry's spread across fifteen real infection traces and derives its basic reproduction number in closed form.

The quantity that governs whether an outbreak takes off or dies is the **basic reproductive number** R_0: the expected number of new hosts a single infected host goes on to infect. The epidemic threshold is the familiar one,

$$R_0 > 1 \;\Rightarrow\; \text{outbreak grows;} \qquad R_0 < 1 \;\Rightarrow\; \text{outbreak dies out.}$$

For a scanning worm, R_0 factors, roughly, into

$$R_0 \approx \underbrace{\beta}_{\text{scan rate}} \times \underbrace{\tau}_{\text{infectious lifetime}} \times \underbrace{\frac{S}{N}}_{\text{susceptible fraction}} \times \underbrace{p}_{\text{per-contact exploit success}} .$$

Every historically successful defense is an attack on one of these terms. **Patching** shrinks the susceptible fraction S/N. **Firewalls and segmentation** cut the effective scan rate β. **Detection and remediation** shorten the infectious lifetime τ. **Credential hygiene** lowers the per-contact success p. Push any of these hard enough and R_0 crosses below 1, and the outbreak collapses on its own.

The crucial historical fact is that the p and S/N terms are **coupled to a fixed exploit set**. A traditional worm carries a finite library of vulnerabilities. Once those are patched, S/N

collapses toward zero *for that worm*, and no amount of scanning speed can rescue R_0. This is why worms become obsolete: not because they are detected, but because the vulnerable population they can reproduce into disappears. The exploit is a *depleting resource*.

4 Stealth: attacking the infectious-lifetime term

There is a second term defenders can be *forced* to concede, and it is what *How to 0wn the Internet* is really about: the infectious lifetime τ. A worm stays infectious only until it is noticed and remediated. Everything a worm does to avoid being noticed is, in the epidemiological accounting, an attack on τ — the term an adaptive adversary can push on *without* discovering a single new vulnerability.

Slammer burned out because it was maximally loud: random-address scanning at bandwidth speed lights up every telescope, honeypot, and rate-based sensor, so τ was minutes. Loud-and-fast maximizes β but minimizes τ. Staniford, Paxson, and Weaver pointed out — *for defenders* — that this trade-off is not fixed; a worm could choose a very different point on the β–τ curve. Their taxonomy of quieter strategies, formalized further in Weaver et al.'s taxonomy of worms [9], is worth stating conceptually because each implies a specific place to look:

- **Hit-list / pre-computed targeting.** A pre-seeded list of reachable hosts avoids the noisy reconnaissance sweep of a random scanner. *Detection surface:* the list must be built somewhere, sometime — the reconnaissance is displaced, not eliminated. The theoretical top speed of such "flash" worms was worked out in [10].
- **Topological / contagion spread.** The worm propagates only along the host's *existing* trust and communication paths, so per-flow its traffic looks legitimate. *Detection surface:* it still perturbs the timing, volume, and content *distribution* of those channels — exactly what communication-graph and flow-anomaly detection is built to catch.
- **Low-and-slow / sub-threshold pacing.** Spread deliberately slowly, under whatever rate-based alarm the environment uses. *Detection surface:* self-limiting, which is the whole point below.

The Intelligent Worm angle is the same thesis on a different term. Instead of choosing a point on the β–τ curve at authoring time, an onboard loop could *pace itself adaptively*, inferring the local detection posture and throttling propagation to whatever the environment tolerates. But **stealth is not free in the R_0 arithmetic**. Every quiet strategy buys τ by paying β, and R_0 is their product. A worm that paces below your detection threshold has, by definition, throttled its own reproductive rate; drive stealth far enough and β falls until $R_0 < 1$ on its own. The concrete defensive implication: *lowering your detection threshold does not just catch the worm faster — it directly compresses the β the attacker is allowed to use*, shrinking the entire region of the curve where the worm can survive. The countermeasure to stealth is not better signatures (a contagion worm has none) but **behavioral and communication-graph anomaly detection**.

5 A history of static worms

Worm	Year	Lesson for defenders
Morris	1988	No infection check ⇒ runaway re-infection that DoS'd its own hosts. Led to the founding of CERT.
Code Red / Slammer	2001–03	Slammer fit in one UDP packet and infected ~90% of vulnerable hosts in 10 minutes. Connectionless worms spread at near-bandwidth speed.
Conficker	2008	Multi-vector resilience (RCE + USB + weak shares). One patched vector is not enough.
Stuxnet	2010	Worms as precision weapons; air gaps are not sufficient isolation.
WannaCry	2017	A propagation engine bolted onto ransomware; global spread in hours — for a vulnerability patched two months earlier.

The consistent pattern: **worms exploit known, unpatched vulnerabilities far more often than novel ones.** That single empirical regularity is why patch management is the highest-leverage defense in existence — it directly depletes S/N, the term the attacker cannot manufacture more of. *The attacker cannot manufacture more susceptible hosts* — under the static assumption. The Intelligent Worm is a hypothesis about relaxing exactly that constraint.

6 The generational shift

Generation	Exploit source	Adaptation	Human?
First (Morris→Slammer)	Static library baked in at authoring	None	Yes
Second (modular / botnet)	Downloaded modules from C2	Human-created updates	Occasionally
Third (*hypothetical*)	Reasoning loop synthesizes candidates	Autonomous, in-the-field	Minimal

The jump that matters is between the second and third rows, and it is subtler than "the AI writes the exploits." Modular worms already decouple propagation from exploitation. What is new is **who closes the loop**. In the modular generation, when defenders patch a vector, a *human* must notice, discover a new vector, write a module, and push it — and that human is both the rate limiter and the thing defenders can target. The Intelligent Worm's claim is that the loop closes *onboard*, with no human in the path. Formally, the shift is from a fixed per-contact success probability p to a **time-varying, self-updating** $p(t)$ that the adversary drives back up as defenders drive it down. The epidemic threshold stops being a one-time cliff and becomes a *contested equilibrium* — a genuinely different control problem.

7 The adaptive loop, at the architectural level

Stripped of offensive specifics, the proposed mechanism is an autonomous control loop — an OODA cycle, or a perception–planning–action loop — wrapped around the classic propagation engine:

```
      |
      v
Generate candidate capabilities   (reasoning engine: LLM + tools)
      |
      v
Evaluate in a safe validation sandbox
      |
      v
Select an effective capability
      |
      v
Update the internal exploitation module
      |
      v
Continue propagation logic  --> (loop back to Observe)
```

The critical, easily-missed box is *evaluate in a safe validation sandbox*. An adaptive worm cannot trust its own generated code any more than the rest of us can trust an LLM's first draft; without an onboard verification step it would mostly generate confident nonsense and announce its presence by crashing targets. Verification is the feasibility bottleneck — and, as we will see, a defensive gift.

A biological analogy, stated precisely so it is not overread: a traditional worm is a **clonal organism** that reproduces without variation, so when patches deploy the population declines. An adaptive worm is closer to an organism that can **generate novel traits under selection pressure**. This describes the *dynamics*, not literal biological evolution. But the epidemiological consequence rhymes: a pathogen that can vary its infection mechanism faster than the host population can immunize is a qualitatively harder containment problem.

8 The full capability stack — and the footprint each subsystem adds

A serious threat model has to reason about the whole system. Take each capability an adversary would want, place it in the R_0 framework, and name the **detection surface it necessarily creates**. The recurring finding: every capability that helps the worm reproduce is also a *behavior that must execute*, and behavior is the thing defenders can see.

- **Centralized exploit generation (offloaded to a GPU back end).** The heaviest part of the loop — reasoning over binaries and protocols, fuzzing, synthesizing and validating exploits — is compute-hungry and noisy, exactly what you would *not* run on a compromised endpoint. The realistic design offloads it to central infrastructure and pushes vetted modules back down. *Defensive read:* this **re-centralizes** what full autonomy tried to distribute, creating a command-and-control dependency to detect, sinkhole, and disrupt. The WannaCry kill-switch lesson generalizes.
- **Adaptive propagation pacing.** The worm profiles link capacity, host performance, and observed monitoring posture, speeding up on fat unmonitored pipes and crawling on instrumented segments. *Defensive read:* the trade-off floor still binds, and the *profiling itself* is anomalous behavior a sensor can flag. The worm runs slowest exactly where defenders instrument most.
- **Inter-worm coordination (avoiding double-infection).** Morris's defining failure was the absence of an infection check. A capable worm wants an **infection marker** to skip already-compromised hosts. *Defensive read:* this is the cleanest defensive judo in the stack — an infection marker is a signature, and it can be *inverted into a vaccine*: mark healthy hosts and the worm skips them. The coordination channel is itself detectable.

- **Persistence and concealment (rootkit-class hiding).** To extend τ, the worm hides from the operator and from tooling. *Defensive read:* hiding is not the absence of behavior; it is *additional* behavior. Tampering with the kernel, hooking system calls, or forging boot state are detectable through integrity monitoring, introspection, memory forensics, and a **hardware root of trust** a software rootkit cannot forge. Concealment raises the cost of detection; it does not eliminate the surface.
- **Cross-platform reach (Linux / macOS / Windows).** Portability raises the effective S/N by widening the reachable host set. *Defensive read:* portability is bought with generality and bulk — larger code, more carried runtime — which *increases* the observable footprint. Behavioral detection built on *what the worm does* covers all platforms at once.

The synthesis cuts against reading the list as a shopping cart. Each capability is a **subsystem that must act**, and every action is a place to be seen. A worm with all five is not stealthier; it has a *larger* aggregate behavioral signature layered on a central dependency defenders can target. Sophistication and detectability grow together — a fact the defense should exploit rather than fear.

9 Feasibility: taking the limits seriously

It would be dishonest to present this as a near-term capability. A language model *alone* is not going to autonomously discover reliable zero-days and consistently produce working exploits against arbitrary targets. The binding constraints are real: hallucination and brittle generated code; the cost of verifying an exploit works *without* tripping monitoring; the difficulty of autonomous experimentation on live or faithfully-reconstructed targets; and weak long-horizon memory and planning.

What current research shows is more modest and more interesting than "AI writes worms." LLM *agents* — models wrapped in tool-use scaffolding — can perform pieces of the offensive chain under favorable conditions. Morris II [22] demonstrated self-replicating adversarial prompts propagating through GenAI-powered applications, building on indirect prompt injection [23] and paralleled by self-replicating prompts across multi-agent systems [24]. The Fang et al. line argued LLM agents can autonomously hack websites [15], exploit one-day vulnerabilities [16], and, in teams, make progress on zero-day-style tasks [17] — all heavily caveated by scaffolding, known vulnerability classes, and web targets. A parallel line of penetration-testing agents and benchmarks [18, 19, 20, 21] consistently finds that *interactive tooling*, not raw model scale, moves the needle. On the defensive and dual-use side, Google's Big Sleep reported an LLM agent finding a previously unknown SQLite bug in 2024, and DARPA's AI Cyber Challenge pushed automated find-and-patch systems to the DEF CON finals.

The honest synthesis: a realistic adaptive system would *not* be an LLM by itself. It would be an LLM orchestrating the mature machinery of program analysis — fuzzing, symbolic execution, static analysis, dynamic instrumentation, automated test generation. ML has been applied across this pipeline for years (neural-guided fuzzing [25], surveyed in [27]), though the gains are contested enough that careful re-evaluations [26] find them smaller than first reported. The model would supply hypothesis formation, cross-domain reasoning, and planning; verification would be delegated to tools that do not hallucinate. That hybrid is the thing worth designing defenses for *now*.

10 The defensive re-architecture

If the exploit term can become adaptive — even partially, even hybrid — then a posture built around *static* malware is aiming at the wrong invariant. Signatures describe a fixed artifact; an adaptive adversary changes the artifact. The center of gravity must shift from *what the malware is* to *what it does*.

- **Behavior-based over signature-based detection.** Behaviors — mass scanning, lateral movement, anomalous process trees, and the worm's own validation activity — are far harder to disguise than a regenerating artifact. Detectors aimed at self-propagating malware already work this way: port-level network profiling [11] flags worm-like traffic early, network reachability structure governs how far an outbreak gets [12], and even signature-generation defenses have been analyzed for how fast they converge to contain a novel worm [13].
- **Runtime isolation and least privilege.** Shrink what a foothold can reach; an adaptive worm that cannot touch new subnets cannot drive $p(t)$ back up — you attack β directly.
- **Aggressive segmentation and egress filtering.** Bound the reachable target set, holding down S/N *structurally*, independent of patching.
- **Rapid patch pipelines.** Still the highest-leverage lever — but against an adaptive adversary, patch latency becomes a live variable in a race, not a countdown that always terminates in the defender's favor.
- **AI-assisted defensive agents.** The same autonomous reasoning that could drive adaptive offense is available for continuous monitoring and automated remediation. Whether defense or offense benefits more from AI is the open empirical question of this decade.
- **Kill switches and circuit breakers.** Deliberate rate caps, quarantine triggers, and network fuses are cheap insurance against fast self-propagation.

The unifying observation: against a static adversary, defense is about **detecting a known thing**; against an adaptive one, defense is about **denying the conditions the adaptation depends on** — reachability, privilege, time, and undetected experimentation. Those conditions are architectural, and we can build them *before* the threat matures.

11 Is this a phase transition?

For four decades, defenders have banked on capabilities being static between releases — an assumption baked into signature databases, CVE cadences, and the mental model of "a threat" as a fixed thing you characterize and block. If autonomous systems can continuously vary the exploitation term faster than a population can immunize, the epidemic threshold stops being a cliff and becomes an equilibrium both sides continuously contest, and the discipline must move from *characterize-then-block* to *continuous behavioral monitoring and rapid autonomous response*.

The value of the Intelligent Worm as a construct is not that it is buildable today — the fully autonomous version is not. Its value is as a **stress test**: it takes the one assumption our defenses quietly depend on and asks what breaks if it fails. The answer is that signature-centric detection and patch-eventually-wins both degrade, while behavior-centric detection, structural isolation, and autonomous defense become more central. Those are conclusions defenders can act on now, regardless of when or whether the adaptive adversary shows up. That is what threat models are for: not to predict the attack, but to be ready for the class of attack before it arrives.

References

[1] W. O. Kermack and A. G. McKendrick, "A Contribution to the Mathematical Theory of Epidemics," *Proc. Royal Society A*, 1927.

[2] J. O. Kephart and S. R. White, "Directed-Graph Epidemiological Models of Computer Viruses," *IEEE S&P*, 1991.

[3] A. Chernikova, N. Gozzi, S. Boboila, N. Perra, T. Eliassi-Rad, A. Oprea, "Modeling Self-Propagating Malware with Epidemiological Models," 2022. arXiv:2208.03276.

[4] S. Pappu et al., "Understanding Malware Propagation Dynamics through Scientific Machine Learning," 2025. arXiv:2507.07143.

[5] S. Staniford, V. Paxson, N. Weaver, "How to 0wn the Internet in Your Spare Time," *USENIX Security*, 2002.

[6] D. Moore et al., "Inside the Slammer Worm," *IEEE Security & Privacy*, 2003.

[7] D. Moore, C. Shannon, k. claffy, "Code-Red: a case study on the spread and victims of an Internet worm," *ACM IMW*, 2002.

[8] C. C. Zou, W. Gong, D. Towsley, "Code Red Worm Propagation Modeling and Analysis," *ACM CCS*, 2002.

[9] N. Weaver, V. Paxson, S. Staniford, R. Cunningham, "A Taxonomy of Computer Worms," *ACM WORM*, 2003.

[10] S. Staniford, D. Moore, V. Paxson, N. Weaver, "The Top Speed of Flash Worms," *ACM WORM*, 2004.

[11] T. Ongun et al., "PORTFILER: Port-Level Network Profiling for Self-Propagating Malware Detection," 2021. arXiv:2112.13798.

[12] A. Chernikova et al., "Cyber Network Resilience against Self-Propagating Malware Attacks," 2022. arXiv:2206.13594.

[13] H. Valizadeh, M. van Dijk, "On the Convergence Rates of Learning-based Signature Generation Schemes to Contain Self-propagating Malware," 2019. arXiv:1905.00154.

[14] J. Saxe, K. Berlin, "Deep Neural Network Based Malware Detection Using Two Dimensional Binary Program Features," 2015. arXiv:1508.03096.

[15] R. Fang, R. Bindu, A. Gupta, Q. Zhan, D. Kang, "LLM Agents can Autonomously Hack Websites," 2024. arXiv:2402.06664.

[16] R. Fang, R. Bindu, A. Gupta, D. Kang, "LLM Agents can Autonomously Exploit One-day Vulnerabilities," 2024. arXiv:2404.08144.

[17] Zhu et al., "Teams of LLM Agents can Exploit Zero-Day Vulnerabilities," 2024. arXiv:2406.01637.

[18] A. Happe, J. Cito, "Getting pwn'd by AI: Penetration Testing with Large Language Models," 2023. arXiv:2308.00121.

[19] G. Deng et al., "PentestGPT: An LLM-empowered Automatic Penetration Testing Tool," 2023. arXiv:2308.06782.

[20] T. Abramovich et al., "EnIGMA: Interactive Tools Substantially Assist LM Agents in Finding Security Vulnerabilities," 2024. arXiv:2409.16165.

[21] L. Muzsai, D. Imolai, A. Lukács, "HackSynth: LLM Agent and Evaluation Framework for Autonomous Penetration Testing," 2024. arXiv:2412.01778.

[22] S. Cohen, R. Bitton, B. Nassi, "Here Comes The AI Worm: Unleashing Zero-click Worms that Target GenAI-Powered Applications" (Morris II), 2024. arXiv:2403.02817.

[23] K. Greshake et al., "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," 2023. arXiv:2302.12173.

[24] D. Lee, M. Tiwari, "Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems," 2024. arXiv:2410.07283.

[25] D. She et al., "NEUZZ: Efficient Fuzzing with Neural Program Smoothing," 2018. arXiv:1807.05620.

[26] M. Nicolae, K. Eisele, A. Zeller, "Revisiting Neural Program Smoothing for Fuzzing," 2023. arXiv:2309.16618.

[27] G. Saavedra, K. Rodhouse, D. Dunlavy, P. Kegelmeyer, "A Review of Machine Learning Applications in Fuzzing," 2019. arXiv:1906.11133.